

Q1: What is a Class in OOP? Give a real-life example.

A class is a blueprint or template used to create objects. It defines variables (data members) and methods (functions). Example: A Car class may have properties like color, speed and methods like drive() and brake().

Q2: What is an Object? How is it different from a Class?

An object is a real-world entity created from a class. Class is a blueprint, whereas object is an instance. Example: Car is a class, BMW is an object.

Q3: What is Encapsulation? Explain with a real-life example.

Encapsulation means wrapping data and methods together and restricting access using access modifiers. Example: Bank account where balance is private.

Q4: What is Abstraction? Give a real-life example.

Abstraction hides internal implementation and shows only essential features. Example: ATM machine.

Q5: What is Inheritance? Why is it used?

Inheritance allows one class to acquire properties of another. It is used for code reusability and better organization.

Q6: What is Polymorphism? Explain with a real-life example.

Polymorphism means one name having many forms. Example: A person behaves differently in different roles.

Q7: What is Association in OOP? Give an example.

Association is a relationship between two classes. Example: Teacher and Student.

Q8: Differentiate between Aggregation and Composition.

Aggregation is a weak relationship, while composition is a strong relationship.

Q9: What is Message Passing in OOP?

Message passing is communication between objects using method calls.

Q10: Why is Java called Platform Independent?

Java is platform independent because it runs on JVM, not directly on OS.

Q11: Explain OOP vs Procedural Programming with examples.

OOP is based on objects (Java), while procedural programming is based on functions (C).

Q12: Explain JVM and Bytecode in detail.

JVM executes bytecode and converts it into machine code. Bytecode is intermediate code.

Q13: Explain Access Specifiers in Java with examples.

Access specifiers include public, private, protected, and default.

Q14: Explain Constructors in Java with types and examples.

Constructors initialize objects. Types include default and parameterized.

Q15: Explain 'this' and 'super' keywords with examples.

'this' refers to current object, 'super' refers to parent class.

Q16: Explain static and final keywords with examples.

'static' is shared among objects, 'final' is constant.

Q17: Explain String, StringBuffer, and StringBuilder.

String is immutable, StringBuffer and StringBuilder are mutable.

Q18: Explain Method Overloading and Method Overriding with examples.

Overloading is compile-time, overriding is runtime.

Q19: Explain Types of Inheritance in Java with examples.

Types include single, multilevel, and hierarchical inheritance.

Q20: Explain Abstract Class vs Interface.

Abstract class has both methods, interface mainly abstract methods.

Q21: Explain Exception Handling in Java with examples.

Exception handling uses try-catch to manage runtime errors.

Q22: Explain Custom Exception with example.

Custom exceptions are user-defined exceptions.

Q23: Explain Multithreading in Java with example.

Multithreading allows multiple threads to run simultaneously.

Q24: Explain Thread Life Cycle.

States include new, runnable, running, waiting, terminated.

Q25: Explain Synchronization and Deadlock.

Synchronization controls access, deadlock blocks threads.

Q26: Explain Inter-thread Communication.

Threads communicate using wait(), notify().

Q27: Explain Applet Life Cycle.

Includes init(), start(), stop(), destroy().

Q28: Explain Dynamic Method Dispatch with example.

Method call resolved at runtime.

Q29: Explain == vs .equals() with example.

'==' compares references, equals() compares values.

Q30: Explain *sleep()* vs *wait()*.

sleep pauses thread, wait releases lock.